

Tuples – Creating tuples, Accessing elements, Understanding immutability, Slicing, Unpacking, and Iterating over tuples.

A tuple is an ordered, immutable collection of items. Tuples are written using parentheses ().

1. Creating Tuples

We can create tuples in several ways.

- Using parentheses
`numbers = (10, 20, 30, 40)`
- Without parentheses
`fruits = 'apple', 'banana', 'mango'`
- Single element tuple (comma is mandatory)
`one = (5,)`
- Empty tuple
`empty = ()`

Example :

`numbers = (10, 20, 30, 40)`

`fruits = 'apple', 'banana', 'mango'`

`one = (5,)`

`empty = ()`

Output / Type

<code>(10, 20, 30, 40)</code>	<code><class 'tuple'></code>
<code>('apple', 'banana', 'mango')</code>	<code><class 'tuple'></code>
<code>(5,)</code>	<code><class 'tuple'></code>
<code>()</code>	<code><class 'tuple'></code>

2. Accessing Elements

Elements of a tuple are accessed using index.

Index starts from 0.

Example :

`t = ('p', 'y', 't', 'h', 'o', 'n')`

Index	0	1	2	3	4	5
Value	'p'	'y'	't'	'h'	'o'	'n'

Accessing :

- `t[0] → 'p'` # first element
- `t[3] → 'h'` # element at index 3
- `t[-1] → 'n'` # last element
- `t[-2] → 'o'` # second last element

3. Understanding Immutability

Tuples are immutable, meaning we cannot change, add or remove elements once the tuple is created.

Example :

`t = (10, 20, 30)`

`t[0] = 100` # `TypeError`

`t.append(40)` # `AttributeError`

`del t[1]` # `TypeError`

Note :

We can reassign the whole tuple, but not modify individual elements.

Example :

`t = (10, 20, 30)`

`t = (40, 50)` # valid

4. Slicing Tuples

We can slice a tuple same as list.

Example :

`t = (10, 20, 30, 40, 50, 60)`

Expression	Result (Tuple)
<code>t[1:4]</code>	<code>(20, 30, 40)</code>
<code>t[:3]</code>	<code>(10, 20, 30)</code>
<code>t[3:]</code>	<code>(40, 50, 60)</code>
<code>t[:2]</code>	<code>(10, 30, 50)</code>
<code>t[::-1]</code>	<code>(60, 50, 40, 30, 20, 10)</code>

Note :

Slicing returns a new tuple. Original tuple remains unchanged.

5. Unpacking Tuples

We can unpack a tuple into multiple variables.

Example :

`t = ('a', 'b', 'c')`

`x, y, z = t`

`print(x, y, z)` # Output: a b c

Unpacking with * (for extra values)

`t = (1, 2, 3, 4, 5)`

`a, b, *rest = t`

`print(a)` # 1

`print(b)` # 2

`print(rest)` # [3, 4, 5]

Swapping using unpacking

`a, b = 10, 20`

`a, b = b, a`

`print(a, b)` # Output: 20 10

6. Iterating Over Tuples

We can iterate over tuples using loops.

Example :

`t = ('red', 'green', 'blue')`

`for color in t:`

`print(color)`

Output :

- red
- green
- blue

Summary

- Tuples are ordered and immutable.
- We access elements using index.
- Slicing returns a new tuple.
- We can unpack tuple values into variables.
- We can iterate over tuples using loops.



Tuples - Creating tuples.

A **tuple** is an ordered, immutable collection of items written inside parentheses ().

1. Creating Tuples

We can create tuples in several ways.

(a) Using parentheses (most common way)

```
t = (10, 20, 30, 40) # creates a tuple with 4 elements
```

(b) Without parentheses

```
t = 10, 20, 30, 40 # parentheses are optional
```

(c) Single element tuple

```
t = (5,) # comma is mandatory for single element tuple
```

(d) Empty tuple

```
t = () # empty tuple
```

Key Points

- Tuple elements are written inside parentheses ().
- Tuples are ordered.
- Tuples are immutable (cannot be changed).
- Duplicate values are allowed.

2. Examples

Code	Tuple Created	Type
t1 = (1, 2, 3, 4)	(1, 2, 3, 4)	<class 'tuple'>
t2 = 'a', 'b', 'c'	('a', 'b', 'c')	<class 'tuple'>
t3 = (100,)	(100,)	<class 'tuple'>
t4 = ()	()	<class 'tuple'>

3. Example Program

```
# Creating tuples
t1 = (10, 20, 30)
t2 = 'apple', 'banana', 'mango'
t3 = (5,) # single element tuple
t4 = () # empty tuple
print(t1, type(t1))
print(t2, type(t2))
print(t3, type(t3))
print(t4, type(t4))
```

Output

```
(10, 20, 30) <class 'tuple'>
('apple', 'banana', 'mango') <class 'tuple'>
(5,) <class 'tuple'>
() <class 'tuple'>
```

Tuples – Accessing Elements

Elements of a tuple are accessed using index.

Indexing in Python starts from 0.

1. Accessing Elements

Example :

```
t = ('red', 'green', 'blue', 'yellow')
```

Index	0	1	2	3
Value	'red'	'green'	'blue'	'yellow'

Accessing :

```
t[0] → 'red'    # first element
t[1] → 'green'   # element at index 1
t[2] → 'blue'    # element at index 2
t[3] → 'yellow'  # last element
```

2. Negative Indexing

We can also access elements using negative index.

Index	-4	-3	-2	-1
Value	'red'	'green'	'blue'	'yellow'

Accessing with Negative Index :

```
t[-4] → 'red'
t[-3] → 'green'
t[-2] → 'blue'
t[-1] → 'yellow'
```

3. Example Program

```
# Accessing elements of a tuple
t = ('red', 'green', 'blue', 'yellow')
print(t[0])      # red
print(t[1])      # green
print(t[2])      # blue
print(t[3])      # yellow
print(t[-1])     # yellow
print(t[-3])     # green
```

Output :

```
red
green
blue
yellow
yellow
green
```

Key Points

- Use index to access tuple elements.
- Positive index starts from 0.
- Negative index starts from -n to -1.

Tuples – understanding immutability

Tuples are **immutable**, meaning once a tuple is created, its elements cannot be changed, added or removed.

Key idea :

You can access tuple elements, but you cannot modify them after creation.

Example 1 : Valid operations (accessing)

```
t = (10, 20, 30, 40)
print(t[0])      # 10
print(t[2])      # 30
print(t[-1])     # 40
print(len(t))    # 4
```

← Accessing elements is allowed.

Example 2 : Invalid operations (modifying)

```
t = (10, 20, 30, 40)

t[1] = 200      # Error
TypeError: 'tuple' object does not
support item assignment

t.append(50)     # Error
AttributeError: 'tuple' object has
no attribute 'append'

del t[2]         # Error
TypeError: 'tuple' object doesn't
support item deletion

t[1:3] = (200, 300) # Error
TypeError: 'tuple' object does not
support item assignment
```

← Any attempt to modify a tuple raises an error.

Example 3 : Creating a new tuple (allowed)

```
t = (10, 20, 30, 40)
t2 = t + (50,)      # concatenation
t3 = t[:2] + (100,) + t[2:] # slicing
print(t2)           # (10, 20, 30, 40, 50)
print(t3)           # (10, 20, 100, 30, 40)
```

Instead of modifying, we create a new tuple.

Key Points

- Tuples are immutable (cannot be changed).
- Elements cannot be assigned, deleted or modified.
- New tuples can be created using concatenation or slicing.
- Immutability makes tuples safe and faster.

Summary

- Lists are mutable (can change), but tuples are immutable (cannot change).
- Use tuples when data should remain constant.

Tuples - Slicing

Slicing allows us to extract a part of a tuple. The sliced part is returned as a **new** tuple.

1. Syntax

`t[start : stop : step]`

Where, start → starting index (default is 0)

stop → ending index (not included)

step → step size (default is 1)

2. Example Tuple

Let us consider a tuple `t = (10, 20, 30, 40, 50, 60)`

Index	0	1	2	3	4	5
Value	10	20	30	40	50	60

3. Slicing Examples

Expression	Result (Tuple)	Explanation
<code>t[1:4]</code>	<code>(20, 30, 40)</code>	Elements from index 1 to 3
<code>t[:4]</code>	<code>(10, 20, 30, 40)</code>	From beginning to index 3
<code>t[2:]</code>	<code>(30, 40, 50, 60)</code>	From index 2 to the end
<code>t[:]</code>	<code>(10, 20, 30, 40, 50, 60)</code>	Entire tuple (a copy)
<code>t[::2]</code>	<code>(10, 30, 50)</code>	Every 2nd element
<code>t[1::2]</code>	<code>(20, 40, 60)</code>	Start at index 1, every 2nd element
<code>t[::-1]</code>	<code>(60, 50, 40, 30, 20, 10)</code>	Reverse of the tuple
<code>t[4:1:-1]</code>	<code>(50, 40, 30)</code>	From index 4 down to index 2

Key Points

- stop index is not included.
- Slicing does not modify the original tuple.
- Slicing always returns a new tuple.

Example Program

```
t = (10, 20, 30, 40, 50, 60)
```

```
print(t[1:4]) # (20, 30, 40)
```

```
print(t[:3]) # (10, 20, 30)
```

```
print(t[3:]) # (40, 50, 60)
```

```
print(t[::-1]) # (60, 50, 40, 30, 20, 10)
```

```
print(t[::2]) # (10, 30, 50)
```

Original tuple
remains
unchanged

`t = (10, 20, 30, 40, 50, 60)`

Tuples - Unpacking

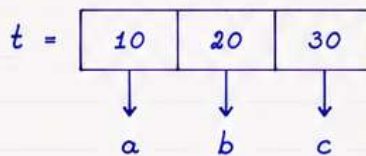
Unpacking allows us to extract values from a tuple and assign them to multiple variables in a single statement.

1. Basic Unpacking

The number of variables must match the number of elements in the tuple.

Example :

```
t = (10, 20, 30)
a, b, c = t
print(a)      # 10
print(b)      # 20
print(c)      # 30
```



Key Points

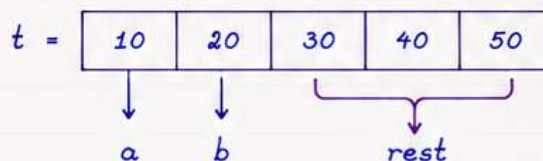
- Unpacking makes code clean and readable.
- The number of variables must match the number of tuple elements.
- Useful for swapping values and returning multiple values from functions.

2. Using * for Extra Elements

We can use `*` to collect remaining elements into a list.

Example :

```
t = (10, 20, 30, 40, 50)
a, b, *rest = t
print(a)      # 10
print(b)      # 20
print(rest)   # [30, 40, 50]
```



3. Swapping Variables

Unpacking is often used to swap values.

Example :

```
a = 10
b = 20
a, b = b, a      # unpacking
print(a)         # 20
print(b)         # 10
```

4. Ignoring Values

Use `_` to ignore unwanted values.

Example :

```
t = (100, 200, 300)
_, b, _ = t
print(b)      # 200
```

5. Function Returning Multiple Values

A function can return a tuple, which can be unpacked.

Example :

```
def get_person():
    return ('Alice', 25, 'Engineer')
```



```
name, age, profession = get_person()
print(name)           # Alice
print(age)            # 25
print(profession)     # Engineer
```

Output :

Alice
25
Engineer

Tuples – iterating

Iteration means accessing each element of a tuple one by one.

Since tuples are immutable, we cannot modify them during iteration, but we can read each element.

1. Iterating using a for loop

We can use a for loop to access each element of a tuple.

Example :

```
t = (10, 20, 30, 40)
for item in t:
    print(item)    # Output
```

Output : 10
20
30
40

2. Iterating using index and range()

We can also iterate over a tuple using index and range().

Example :

```
t = (10, 20, 30, 40)
for i in range(len(t)):
    print(t[i])    # Output
```

Output : 10
20
30
40

3. Using enumerate()

We can get both index and value using enumerate().

Example :

```
t = ('red', 'green', 'blue')
for index, value in enumerate(t):
    print(index, '->', value) # Output
```

Output : 0 -> red
1 -> green
2 -> blue

4. Iterating with tuple unpacking

If a tuple contains tuples, we can iterate using unpacking.

Example :

```
t = ((1, 'A'), (2, 'B'), (3, 'C'))
for num, ch in t:
    print(num, ': ', ch)    # Output
```

Output : 1 : A
2 : B
3 : C

Key Points

- We use for loops to iterate over tuples.
- Tuples cannot be changed during iteration.
- We can use range(), enumerate(), or unpacking for advanced iteration.
- Iteration helps us process each element of a tuple efficiently.